

C COMPILER PROJECT

C Compiler Front-End Language & Semantic Specification

Frozen Specification - Version 1.0

Item	Decision
Project	Design and Implementation of a C Compiler
Implementation Language	C
Input Language	C - defined project subset
Input File Extension	.c
Current Compiler Scope	Front end through semantic analysis
Specification Status	Frozen baseline for implementation

Purpose: This document is the single reference specification for the compiler front end. It defines what the compiler accepts, rejects, and reports before implementation begins.

Contents

1. Document Purpose and Scope
2. Compiler Front-End Architecture
3. Design Principles and Scope Boundaries
4. Supported Source Program Structure
5. Lexical Specification
6. Keywords and Reserved Words
7. Identifiers
8. Literals and Strings
9. Data Types
10. Operators and Precedence
11. Delimiters and Special Symbols
12. Comments and Whitespace
13. Declarations and Variables
14. Arrays
15. Functions and Function Calls
16. Statements and Control Flow
17. Input/Output: printf and scanf
18. Explicit Type Casting
19. Implicit Conversion Rules
20. Expression Type Rules
21. Scope and Symbol-Table Rules
22. Semantic Error Catalogue
23. Grammar Specification
24. AST Requirements
25. Symbol Table Specification
26. Semantic Analyzer Requirements
27. Supported vs Unsupported Feature Matrix
28. Conformance Examples and Test Categories
29. Implementation Roadmap
30. Frozen Decisions and Change Control

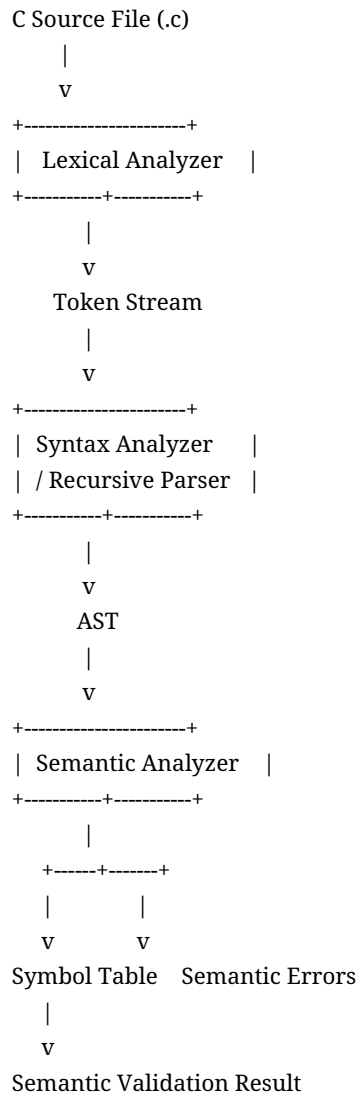
1. Document Purpose and Scope

This document freezes the language, grammar, typing, scope, error, AST, symbol-table, and semantic-analysis rules for the compiler front end. It is intended to be used during implementation, debugging, testing, viva preparation, and later integration with intermediate-code and back-end phases.

The compiler is written in C and accepts source programs in C syntax, but the project deliberately implements a defined subset of C rather than the entire C standard. The subset is intentionally broad enough to express ordinary beginner-to-intermediate programs while excluding features whose implementation would introduce major additional complexity, such as general pointers, structs, unions, preprocessing, and dynamic memory management.

Any construct not explicitly listed as supported in this document is considered unsupported unless this specification is formally amended before implementation of the affected module.

2. Compiler Front-End Architecture



The current front end ends after semantic validation. Later project phases may consume the validated AST and symbol information to generate intermediate representation, optimize it, and produce target code.

3. Design Principles and Scope Boundaries

- C syntax is used wherever practical; the compiler does not introduce a new programming language name.
- The implementation language is C.
- The parser is designed for recursive-descent implementation, so the frozen grammar avoids direct left recursion in its parser-oriented form.
- Semantic rules are intentionally simpler than full ISO C rules when a simpler rule improves predictability and implementability.
- Strings are supported as literals and for formatted I/O, but there is no general-purpose string type or string library in the front-end subset.
- Pointers are not general language values. The ampersand operator is recognized only where required by scanf input arguments.
- Any unsupported feature must fail clearly rather than being silently interpreted as another feature.

4. Supported Source Program Structure

A complete program is composed of external declarations. The entry point is a parameterless `int main()` function.

```
int add(int a, int b) {
    return a + b;
}

int main() {
    int x = add(10, 20);
    printf("x = %d\n", x);
    return 0;
}
```

The semantic analyzer must ensure that `main` is declared as `int main()` and that the program contains exactly one valid entry point.

5. Lexical Specification

The lexer scans source characters and produces tokens with token kind, lexeme, and source location (line and column).

Token Category	Examples / Notes
Keywords	int, float, char, void, main, if, else, while, for, do, return, break, continue
Identifiers	x, total, _count, student_marks

Integer literals	0, 10, 250, 1000
Floating literals	3.14, 0.5, 100.0
Character literals	'A', '5', '\n'
String literals	"Hello", "%d", "Value = %f\n"
Operators	+ - * / % < > <= >= == != && ! = += -= *= /= %= ++ --
Delimiters	() {} [] ; ,
Special input symbol	& for scanf argument syntax

6. Keywords and Reserved Words

The following words are reserved and cannot be used as ordinary identifiers:

```
int
float
char
void
main
if
else
while
for
do
return
break
continue
```

printf and scanf are not keywords. They are predefined function identifiers supplied by the compiler environment.

7. Identifiers

identifier -> (letter | _)(letter | digit | _)*

Valid	Invalid	Reason
x	1x	Identifier cannot begin with a digit
total	total-price	Hyphen is not part of an identifier
_count	int	Keyword cannot be used as identifier
student2	2student	Identifier cannot begin with a digit

8. Literals and Strings

8.1 Integer Literals

Decimal integer literals are supported. The initial implementation does not require hexadecimal, octal, binary, or suffix-based integer forms.

8.2 Floating-Point Literals

The initial form is digits followed by a decimal point and digits, e.g. 3.14 or 100.0. Scientific notation is not required in the frozen baseline.

8.3 Character Literals

A character literal contains one character or one supported escape sequence between single quotes.

```
'A'  
'5'  
'\n'  
'\t'
```

8.4 String Literals

String literals are supported and are important for printf. They may contain ordinary characters and the following initial escape sequences:

```
\n\t\\\"'
```

A general mutable string data type is not introduced. Strings are represented as literals at the front-end interface and may later be mapped to an appropriate representation during code generation.

9. Data Types

Type	Meaning	Variable Use	Notes
int	Signed integer	Allowed	Default integer arithmetic type
float	Floating-point value	Allowed	Supports mixed numeric expressions with int
char	Single character	Allowed	Kept distinct from int in semantic rules
void	No value	Not allowed	Used for function return type only

10. Operators and Precedence

Precedence	Operators	Associativity	Purpose
1 - highest	0 []	Left	Grouping and array

			indexing
2	postfix ++ --	Left	Postfix increment/decrement
3	prefix ++ -- + - ! &	Right	Unary operations; & restricted to scanf destination notation
4	* / %	Left	Multiplicative
5	+ -	Left	Additive
6	< > <= >=	Left	Relational
7	== !=	Left	Equality
8	&&	Left	Logical AND
9		Left	Logical OR
10 - lowest	= += -= *= /= %=	Right	Assignment

Examples: `a + b * c` is interpreted as `a + (b * c)`; `a = b = 10` is interpreted as `a = (b = 10)`.

11. Delimiters and Special Symbols

Symbol	Use
()	Function calls, parameter lists, conditions, grouping
{ }	Blocks
[]	One-dimensional array declaration and indexing
;	Statement terminator
,	Parameter/declaration separator
&	scanf destination notation only

12. Comments and Whitespace

- Whitespace, tabs, and newlines separate lexical units and are otherwise ignored.
- Single-line comments begin with `//` and continue to end of line.
- Multi-line comments begin with `/*` and end with `*/`.
- Comments do not produce tokens. Unterminated multi-line comments are lexical errors.

13. Declarations and Variables

```
int x;
float price;
char grade;
```

```
int a = 10, b = 20;
float average = 12.5;
char ch = 'A';
```

Declarations may appear at global scope and block/function scope, subject to the symbol-table and scope rules. The front end does not support implicit variable declarations.

14. Arrays

One-dimensional arrays are supported.

```
int marks[5];
float values[10];
char letters[26];
```

```
marks[0] = 95;
int i = 2;
marks[i] = 90;
```

- Array size is a positive integer literal in the baseline declaration syntax.
- Array indices must have integer type.
- Array element assignment follows the element type conversion rules.
- Multidimensional arrays are not supported.
- Dynamic allocation and pointer-based array manipulation are not supported.

15. Functions and Function Calls

```
int add(int a, int b) {
    return a + b;
}
```

```
float average(float a, float b) {
    return (a + b) / 2;
}
```

```
void display() {
    printf("Hello\n");
    return;
}
```

- Functions may return int, float, char, or void.
- Parameters may have int, float, or char types.
- Functions may call functions defined earlier in the source or visible through the compiler environment.
- The current baseline does not support pointer parameters, function pointers, variadic user-defined functions, or complex declarators.
- main must be exactly `int main()`.

16. Statements and Control Flow

16.1 Declaration and Assignment

```
int x = 10;
x = x + 5;
x += 2;
```

16.2 if / else

```
if (x > 10) {
    x++;
} else {
    x--;
}
```

16.3 while

```
while (x < 100) {
    x++;
}
```

16.4 do-while

```
do {
    x++;
} while (x < 100);
```

16.5 for

```
for (int i = 0; i < 10; i++) {
    printf("%d\n", i);
}
```

16.6 break and continue

```
for (int i = 0; i < 10; i++) {
    if (i == 5) continue;
    if (i == 8) break;
}
```

16.7 return

```
return 0;
return x;
return;
```

17. Input/Output: printf and scanf

17.1 printf

Specifier	Required Argument Type
%d	int
%f	float

%c	char
%s	string literal

The semantic analyzer should match each supported format specifier with the corresponding argument type and report mismatches. For %s, the argument must be a string literal in the frozen baseline because no general-purpose string variable type is defined.

```
int x = 10;
float y = 3.14;
char c = 'A';
printf("x=%d y=%f c=%c\n", x, y, c);
printf("Hello\n");
```

17.2 scanf

Specifier	Destination Type
%d	int
%f	float
%c	char

```
int x;
float y;
char c;
scanf("%d", &x);
scanf("%f", &y);
scanf("%c", &c);
```

General pointer semantics are not introduced by scanf. The ampersand form is recognized as a special input-destination construct only: &identifier. The semantic analyzer must reject address-of usage anywhere else.

18. Explicit Type Casting

C-style explicit casts are supported for int, float, and char.

```
float x = 3.14;
int a = (int)x;

int n = 65;
float f = (float)n;
char c = (char)n;
```

An explicit cast is a programmer-requested conversion. It is therefore legal in situations where the corresponding implicit conversion is forbidden, subject to the supported cast matrix.

19. Implicit Conversion Rules

From	To	Implicitly Allowed?	Reason
------	----	---------------------	--------

int	int	Yes	Same type
int	float	Yes	Widening numeric conversion
int	char	No	Cross-category conversion requires explicit cast
float	int	No	Potential narrowing conversion
float	float	Yes	Same type
float	char	No	Cross-category conversion
char	int	No	char is kept distinct in this project
char	float	No	char is kept distinct in this project
char	char	Yes	Same type
void	any value type	No	void does not produce a value

Thus `float x = 10;` is valid, while `int x = 10.5;` is invalid unless an explicit cast is used: `int x = (int)10.5;`

20. Expression Type Rules

20.1 Arithmetic

Left Type	Operator	Right Type	Result Type
int	+ - * /	int	int
int	+ - * /	float	float
float	+ - * /	int	float
float	+ - * /	float	float
int	%	int	int

The modulus operator is restricted to `int` operands. `char` is not automatically promoted for arithmetic.

20.2 Relational and Equality

Allowed comparisons: `int-int`, `int-float`, `float-int`, `float-float`, and `char-char`. The result is an internal boolean value.

20.3 Logical Operators

The operands of `&&`, `||`, and `!` must be boolean-valued relational/equality/logical expressions. General C-style truthiness of arbitrary numeric expressions is not part of the baseline semantic rules.

20.4 Increment/Decrement

Both prefix and postfix forms are supported: `++x`, `--x`, `x++`, and `x--`. `++` and `--` are allowed on modifiable int and float variables or array elements. They are not allowed on char values or non-lvalues.

20.5 Compound Assignment

The final assigned value must be compatible with the left-hand type using the same implicit conversion rules as ordinary assignment. Thus `float x; x += 2` is valid, while `int x; x += 2.5` is invalid.

21. Scope and Symbol-Table Rules

Rule	Required Behaviour
Declaration before use	An identifier must be declared in an accessible scope before use.
No duplicate in same scope	Declaring the same identifier twice in one scope is an error.
Nested scopes	Inner blocks may see outer declarations, subject to shadowing policy.
Out-of-scope use	An identifier outside its declaration scope is an error.
Function parameters	Parameters enter the function scope before body analysis.
Functions	Function names occupy the program/global symbol namespace.
Predefined I/O	<code>printf</code> and <code>scanf</code> are inserted into the initial environment.

Recommended baseline: allow inner-block shadowing of an outer variable, but reject duplicate declarations within the same scope. This keeps scope checking predictable while matching common block-scoped language behaviour.

22. Semantic Error Catalogue

ID	Error Category	Example	Expected Diagnostic
SEM-01	Undeclared identifier	<code>x = 10;</code>	Identifier 'x' is not declared
SEM-02	Duplicate declaration	<code>int x; int x;</code>	Duplicate declaration of 'x'

SEM-03	Out of scope	use y after inner block ends	Identifier 'y' is out of scope
SEM-04	Assignment mismatch	int x = 3.14;	Cannot assign float to int
SEM-05	Invalid arithmetic operands	char + int	Invalid operand types for arithmetic
SEM-06	Invalid modulus	10.5 % 2	Modulus requires int operands
SEM-07	Invalid condition	if (x + 1)	Condition must produce boolean result
SEM-08	Invalid logical operand	x && y	Logical operator requires boolean operands
SEM-09	Wrong function arity	add(1)	Wrong number of arguments
SEM-10	Wrong argument type	int function called with float	Argument type mismatch
SEM-11	Invalid return type	int f(){ return 3.14; }	Return expression incompatible with function type
SEM-12	Missing/invalid return	non-void function without valid return	Non-void function requires compatible return
SEM-13	break outside loop	break; in main body	break is only valid inside a loop
SEM-14	continue outside loop	continue; outside loop	continue is only valid inside a loop
SEM-15	Invalid array index	a[2.5]	Array index must be int
SEM-16	Array assignment mismatch	int a[5]; a[0] = 3.14;	Element assignment incompatible with int
SEM-17	printf format mismatch	printf("%d", 3.14);	%d requires int argument
SEM-18	scanf format mismatch	scanf("%d", &f);	%d requires int destination
SEM-19	Invalid cast	unsupported cast form	Cast type combination is not supported
SEM-20	Invalid main	float main()	Entry point must be int main()

23. Grammar Specification

The following grammar is the parser-oriented baseline. It is intended for a recursive-descent implementation. ϵ denotes the empty production.

```

program
  -> external_declaration*

external_declaration
  -> function_definition
  | declaration

function_definition
  -> 'int' 'main' '(' ')' block
  | type identifier '(' parameter_list_opt ')' block

parameter_list_opt
  -> parameter_list
  | ε

parameter_list
  -> parameter (',' parameter)*

parameter
  -> type identifier

block
  -> '{' statement_list '}'

statement_list
  -> statement*

statement
  -> declaration
  | expression_statement
  | if_statement
  | while_statement
  | for_statement
  | do_while_statement
  | return_statement
  | break_statement
  | continue_statement
  | block

declaration
  -> type init_declarator_list ';'

init_declarator_list
  -> init_declarator (',' init_declarator)*

init_declarator
  -> identifier
  | identifier '=' expression
  | identifier '[' integer_literal ']'

type

```

-> 'int' | 'float' | 'char' | 'void'

expression_statement

-> expression ';' | ';' ;

if_statement

-> 'if' '(' condition ')' statement
| 'if' '(' condition ')' statement 'else' statement

while_statement

-> 'while' '(' condition ')' statement

do_while_statement

-> 'do' statement 'while' '(' condition ')' ';' ;

for_statement

-> 'for' '(' for_init ';' condition_opt ';' for_update_opt ')' statement

for_init

-> declaration_no_semicolon
| expression
| ϵ

declaration_no_semicolon

-> type init_declarator (',' init_declarator)*

condition_opt

-> condition
| ϵ

for_update_opt

-> expression
| ϵ

return_statement

-> 'return' expression ';' | 'return' ';' ;

break_statement

-> 'break' ';' ;

continue_statement

-> 'continue' ';' ;

condition

-> expression

expression

-> assignment_expression

assignment_expression
 -> logical_or_expression assignment_tail

assignment_tail
 -> assignment_operator assignment_expression
 | ε

assignment_operator
 -> '=' | '+=' | '-=' | '*=' | '/=' | '%='

logical_or_expression
 -> logical_and_expression ('|' logical_and_expression)*

logical_and_expression
 -> equality_expression ('&&' equality_expression)*

equality_expression
 -> relational_expression (('==' | '!=') relational_expression)*

relational_expression
 -> additive_expression (('<' | '>' | '<=' | '>=') additive_expression)*

additive_expression
 -> multiplicative_expression (('+' | '-') multiplicative_expression)*

multiplicative_expression
 -> unary_expression (('*' | '/' | '%') unary_expression)*

unary_expression
 -> '++' unary_expression
 | '--' unary_expression
 | ('+' | '-' | '!' | '&') unary_expression
 | cast_expression

cast_expression
 -> '(' type ')' cast_expression
 | postfix_expression

postfix_expression
 -> primary_expression postfix_tail*

postfix_tail
 -> '++'
 | '--'
 | '[' expression ']'
 | '(' argument_list_opt ')'

argument_list_opt
 -> argument_list
 | ε

argument_list
-> expression (',' expression)*

primary_expression
-> identifier
| integer_literal
| float_literal
| char_literal
| string_literal
| '(' expression ')'

Semantic rule for for-loops: when the condition is omitted (for (;)), it is treated as an always-true condition. This is a semantic convention rather than a separate user-visible boolean literal.

24. AST Requirements

The parser should construct an AST that preserves semantic structure while omitting unnecessary punctuation-only grammar detail.

AST Node Category	Representative Data
Program	List of external declarations
Function	Name, return type, parameters, body
Block	Ordered list of statements
Declaration	Type, name, optional initializer, optional array metadata
Assignment	Operator, left-hand expression, right-hand expression
If	Condition, then branch, optional else branch
While / DoWhile / For	Loop condition and body; for init/update
Return	Optional return expression
Break / Continue	No child expression
Call	Callee name/expression and arguments
Array Access	Array expression and index expression
Binary Expression	Operator, left child, right child
Unary Expression	Operator and operand
Cast	Target type and operand
Literal	Literal type and value
Identifier	Name

25. Symbol Table Specification

The symbol table is required for semantic analysis. A table entry should contain enough information to validate uses of the identifier.

Field	Purpose
Name	Identifier spelling
Kind	Variable, array, function, parameter, predefined function
Type	int, float, char, void, or applicable composite metadata
Scope	Global, function, or block scope identity
Array metadata	Element type and size when applicable
Function metadata	Return type, parameter list, parameter count

The initial environment should contain printf and scanf signatures. A later back end may use additional symbol attributes, but they are not required for the current front-end semantic pass.

26. Semantic Analyzer Requirements

The semantic analyzer should traverse the AST and produce diagnostics without crashing on invalid input. Recommended checks are grouped below.

- Name resolution: declarations, uses, functions, and predefined functions.
- Scope: nested block visibility, shadowing policy, and out-of-scope references.
- Type checking: declaration initializers, assignments, operators, returns, and function arguments.
- Control-flow context: break/continue only inside loops.
- Function rules: signature matching, argument count, argument types, return type, and main entry-point rules.
- Array rules: one-dimensional declaration, integer index, element type compatibility.
- I/O rules: printf/scanf format-specifier and argument compatibility.
- Cast rules: explicit cast syntax and supported source/target type combinations.

27. Supported vs Unsupported Feature Matrix

Feature	Status	Notes
Variables	SUPPORTED	int, float, char
Functions	SUPPORTED	Basic user-defined functions
Arrays	SUPPORTED	One-dimensional only
if / else	SUPPORTED	Nested forms allowed
while	SUPPORTED	Nested loops allowed
for	SUPPORTED	Declaration or expression init;

		optional condition/update
do-while	SUPPORTED	Standard syntax
break / continue	SUPPORTED	Loop context checked semantically
printf / scanf	SUPPORTED	Limited format checking
Strings	SUPPORTED	String literals and formatted I/O
C-style casts	SUPPORTED	int/float/char combinations per cast matrix
Implicit int -> float	SUPPORTED	Only implicit cross-type conversion
Pointers	NOT SUPPORTED	Except scanf & identifier destination notation
Struct	NOT SUPPORTED	Excluded from baseline
Union	NOT SUPPORTED	Excluded from baseline
Enum	NOT SUPPORTED	Excluded from baseline
Typedef	NOT SUPPORTED	Excluded from baseline
Function pointers	NOT SUPPORTED	Excluded from baseline
Dynamic memory	NOT SUPPORTED	No malloc/calloc/realloc/free
Preprocessor	NOT SUPPORTED	No macro/include processing
Multidimensional arrays	NOT SUPPORTED	Only 1-D arrays
Implementation-specific extensions	NOT SUPPORTED	Not part of frozen baseline

28. Conformance Examples and Test Categories

28.1 Valid Program - Functions, Arrays, Loop, I/O

```

int sum(int a, int b) {
    return a + b;
}

int main() {
    int values[5];
    int i;
    int total = 0;

    for (i = 0; i < 5; i++) {
        values[i] = i * 10;
        total += values[i];
    }

    printf("total = %d\n", total);
    return 0;
}

```

28.2 Valid Program - Type Promotion and Cast

```
int main() {
    int a = 10;
    float b = a;
    int c = (int)3.14;

    printf("b=%f c=%d\n", b, c);
    return 0;
}
```

28.3 Invalid Semantic Examples

```
int main() {
    int x = 3.14;    // float -> int: invalid implicit conversion
    y = 10;          // y undeclared
    break;           // break outside loop
    return 0;
}
```

28.4 Testing Categories

- Lexical: valid tokens, malformed numbers, unterminated strings, unterminated comments, invalid characters.
- Syntax: missing semicolons, unmatched delimiters, malformed expressions, malformed for statements, malformed function definitions.
- Semantic names: undeclared identifiers, duplicate declarations, function lookup.
- Semantic types: assignments, operators, returns, function arguments, printf/scanf, casts.
- Scope: nested blocks, shadowing, out-of-scope use.
- Control flow: valid/invalid break and continue.
- Arrays: valid indexing, float index, wrong element type, out-of-scope array name.
- Integration: complete programs combining functions, arrays, loops, conditions, I/O, and expressions.

29. Implementation Roadmap

Stage	Module(s)	Acceptance Condition
1	Project skeleton + token.h/token.c	All token kinds compile and are centrally defined
2	Lexer	Valid source becomes correct token stream; lexical errors are reported
3	AST	Parser-facing node types and constructors are available
4	Parser - expressions	Precedence and associativity tests pass
5	Parser - statements/functions	Valid C-subset programs parse; syntax errors are diagnosed
6	Symbol table	Declarations, lookups, scopes, and

		function signatures work
7	Semantic analyzer	Type, scope, function, array, loop, and I/O checks work
8	Integration	Complete front-end pipeline runs from .c input to semantic result
9	Regression tests	Supported examples pass; unsupported features fail predictably

Each stage should be committed to Git before the next major stage. The implementation should remain modular, with headers exposing clear interfaces between lexer, parser, AST, symbol table, semantic analyzer, and driver.

30. Frozen Decisions and Change Control

This specification is the baseline for implementation. Changing a language feature after coding begins can affect tokens, grammar, AST nodes, symbol-table entries, semantic rules, test cases, and later compiler phases.

1. Treat this document as the authoritative feature list for the front end.
2. Do not add a new language feature solely because it is easy to code; first update the specification and its grammar/semantic rules.
3. Do not silently change conversion behaviour; update the conversion table and dependent tests together.
4. Every supported feature must have at least one valid example and one relevant invalid/error case where applicable.
5. Every excluded feature should fail clearly and should not be partially parsed as a different feature.
6. Keep the final implementation modular so that future back-end phases can consume the validated AST and symbol information without redesigning the front end.

Appendix A - Quick Reference

Area	Frozen Decision
Implementation language	C
Input	C source files (.c)
Parser approach	Recursive descent
Data types	int, float, char, void
Strings	Supported as literals / formatted I/O
Functions	Supported
Arrays	1-D supported
Control flow	if, else, while, for, do-while, break, continue

I/O	printf, scanf with format/type checking
Casts	Explicit int/float/char casts
Implicit conversion	Only int -> float across different value types
AST	Required
Symbol table	Required
Semantic analysis	Required
Pointers	Excluded except scanf & identifier destination notation
Struct/Union/Enum	Excluded
Preprocessor	Excluded
Dynamic memory	Excluded

Appendix B - Type Conversion Matrix

Source	Target	Implicit	Explicit Cast	Notes
int	int	Yes	Yes	Same type
int	float	Yes	Yes	Only widening implicit conversion
int	char	No	Yes	Explicit cast required
float	int	No	Yes	Explicit cast required
float	float	Yes	Yes	Same type
float	char	No	Yes	Explicit cast required
char	int	No	Yes	char remains distinct implicitly
char	float	No	Yes	char remains distinct implicitly
char	char	Yes	Yes	Same type

Appendix C - Front-End Acceptance Checklist

- The lexer recognizes every token category listed in Sections 5-12 and records line/column information.
- The parser accepts the grammar in Section 23 and rejects malformed constructs with source locations.
- The AST contains the node categories in Section 24.
- The symbol table stores the fields in Section 25 and supports nested scopes.
- The semantic analyzer implements the checks in Section 26 and the error families in Section 22.
- Each supported feature has at least one regression test.
- Each intentionally unsupported feature has a negative test where practical.
- The compiler does not silently reinterpret unsupported syntax as a supported construct.

Appendix D - Implementation Quick Reference

Module	Primary Responsibility	Key Inputs	Key Outputs
Token	Central token/type definitions	Token kinds, lexemes, source location	Token structures
Lexer	Character scanning and tokenization	Source text	Token stream / lexical diagnostics
Parser	Grammar validation and AST construction	Token stream	AST / syntax diagnostics
AST	Semantic program representation	Parser node data	AST nodes and child relationships
Symbol Table	Name/type/scope information	Declarations, scopes, functions	Lookup and insertion results
Semantic Analyzer	Type, name, scope, function, array, loop, I/O checks	AST + symbol table	Semantic diagnostics + validated AST state
Driver	Pipeline orchestration and user-facing result	.c file	Compilation summary and diagnostics

Recommended repository modules: `src/main.c`, `src/lexer.c`, `src/parser.c`, `src/ast.c`, `src/symbol_table.c`, `src/semantic.c`, `src/error.c`, with matching headers under `include/`. Keep tests and example `.c` programs in separate directories.